

BCC202 - Estruturas de Dados I

Aula 21: Pesquisa Sequencial e Pesquisa Binária

Pedro Silva

Universidade Federal de Ouro Preto, UFOP
Departamento de Computação, DECOM
Email: silvap@ufop.edu.br



Conteúdo

Introdução

Pesquisa Sequencial

Pesquisa Binária

Considerações Finais

Bibliografia

Introdução

Pesquisa:

- ▶ Estudo de como recuperar informação a partir de uma grande massa de informação previamente armazenada.
- ▶ A informação é dividida em registros.
- ▶ Cada registro possui uma chave para ser usada na pesquisa.
- ▶ **Objetivo da pesquisa:** Encontrar uma ou mais ocorrências de registros com chaves iguais à chave de pesquisa.
- ▶ **Pesquisa com sucesso X sem sucesso.**

Abordagens de pesquisa em Memória Primária

- ▶ Pesquisa Sequencial.
- ▶ Pesquisa Binária.
- ▶ Árvores de Pesquisa:
 - ▶ Árvores Binárias de Pesquisa.
 - ▶ Árvores AVL.
- ▶ Transformação de Chave (*Hashing*):
 - ▶ Listas Encadeadas.
 - ▶ Endereçamento Aberto.
 - ▶ *Hashing* Perfeito.

Abordagens de pesquisa em Memória Primária

- ▶ Pesquisa Sequencial.
- ▶ Pesquisa Binária.
- ▶ Árvores de Pesquisa:
 - ▶ Árvores Binárias de Pesquisa.
 - ▶ Árvores AVL.
- ▶ Transformação de Chave (*Hashing*):
 - ▶ Listas Encadeadas.
 - ▶ Endereçamento Aberto.
 - ▶ *Hashing* Perfeito.

Algoritmos de Pesquisa - TADs

- ▶ É importante considerar os algoritmos de pesquisa como tipos abstratos de dados (TADs), de tal forma que haja uma independência de implementação para as operações.
- ▶ Operações mais comuns:
 1. Inicializar a estrutura de dados.
 2. Pesquisar um ou mais registros com determinada chave.
 3. Inserir um novo registro.
 4. Retirar um registro específico.

Dicionário

- ▶ Dicionário é um TAD com as operações:
 1. Inicializa.
 2. Pesquisa.
 3. Insere.
 4. Retira.

- ▶ Analogia com um dicionário da língua portuguesa:
 - ▶ Chaves × Palavras.

 - ▶ Registros × Entradas associadas com pronúncia, definição, sinônimos, outras informações.

Pesquisa Sequencial

Pesquisa Sequencial

- ▶ Método de pesquisa **mais simples**: a partir do primeiro registro, pesquise sequencialmente até encontrar a chave procurada; então pare.
- ▶ Armazenamento de um conjunto de registros por meio de um **array**.
- ▶ A Busca (*find*):
 - ▶ Pesquisa retorna o índice do registro que contém a chave x.
 - ▶ Caso não esteja presente, o valor retornado é -1.
 - ▶ A implementação não suporta mais de um registro com uma mesma chave, pois retorna o primeiro encontrado.

TAD Dicionário

```
1 typedef long Chave;  
2  
3 typedef struct {  
4     Chave chave;  
5     /* outros componentes */  
6 } Registro;  
7  
8 typedef struct {  
9     Registro *v;  
10    int n, max;  
11 } Dicionario;
```

```
12 /* inicializa um dicionário */
13 void DicionarioInicio(Dicionario *dicio) {
14     dicio->n = 0;
15     dicio->max = 10;
16     dicio->v = (Registro*) malloc(sizeof(Registro)* dicio->max);
17 }
18
19 /* encontra e retorna o índice da chave x no dicionário */
20 int DicionarioFind(Dicionario *dicio, Chave c) {
21     int i;
22     for (i=dicio->n-1; i>=0; i--)
23         if (dicio->v[i].chave == c)
24             return i;
25     // retorna -1 caso a chave não seja encontrada
26     return -1;
27 }
```

TAD Dicionário Operações II

```
28 /* insere um registro no dicionário */
29 void DicionarioInsere(Dicionario *dicio, Registro *x) {
30     if (dicio->n == dicio->max) {
31         dicio->max *= 2;
32         dicio->v = (Registro*) realloc(dicio->v, sizeof(Registro) * dicio->max
33     );
34 }
35 // n é o tamanho
36 dicio->v[dicio->n++] = x;
37 }
```

Análise Pesquisa Sequencial

Análise:

- ▶ Pesquisa com Sucesso:
 - ▶ melhor caso: $C(n) = 1$.
 - ▶ pior caso: $C(n) = n$.
 - ▶ caso médio: $C(n) = (n + 1)/2$.
- ▶ Pesquisa sem sucesso:
 - ▶ $C(n) = n + 1$.
- ▶ O algoritmo de pesquisa sequencial é a melhor escolha para o problema de pesquisa com $n < 25$.

Pesquisa Binária

Pesquisa Binária

Pesquisa em tabela pode ser mais eficiente se registros forem mantidos em ordem.

Para saber se uma chave está presente na tabela:

1. Compare a chave com o registro que está na posição do meio da tabela.
2. Se a chave é menor então o registro procurado está na primeira metade da tabela
3. Se a chave é maior então o registro procurado está na segunda metade da tabela.
4. Repita até que a chave seja encontrada ou que se constate que a chave não existe na tabela.

Exemplo de Pesquisa Binária pela chave **G**

0	1	2	3	4	5	6	7
A	B	C	D	E	F	G	H

Exemplo de Pesquisa Binária pela chave **G**

0	1	2	3	4	5	6	7
A	B	C	D	E	F	G	H

Exemplo de Pesquisa Binária pela chave **G**

0	1	2	3	4	5	6	7
A	B	C	D	E	F	G	H

Exemplo de Pesquisa Binária pela chave **G**

0	1	2	3	4	5	6	7
A	B	C	D	E	F	G	H

Exemplo de Pesquisa Binária pela chave **G**

0	1	2	3	4	5	6	7
A	B	C	D	E	F	G	H

Exemplo de Pesquisa Binária pela chave **G**

0	1	2	3	4	5	6	7
A	B	C	D	E	F	G	H

Implementação da Pesquisa Binária

```
1 typedef long Chave;
2
3 typedef struct {
4     Chave Chave;
5     /* outros componentes */
6 } Registro;
7
8 typedef struct {
9     Registro *v;
10    int n, max;
11 } Dicionario;
12
13 /* inicializa um dicionário */
14 void DicionarioInicio(Dicionario *dicio) {
15     dicio->n = 0;
16     dicio->max = 10;
17     dicio->v = (Registro*) malloc(sizeof(Registro) * dicio->max);
18 }
```

```
19 /* encontra o índice da chave x no dicionário */
20 int DicionarioFind(Dicionario *dicio, Chave x) {
21     return DicionarioBinaria(dicio, 0, dicio->n-1, x); // dicio->n é o tamanho
22 }
23
24 /* encontra o índice da chave x no dicionário entre esq e dir */
25 int DicionarioBinaria(Dicionario *dicio, int esq, int dir, Chave x) {
26     int meio = (esq+dir)/2;
27
28     if (dicio->v[meio].chave != x && esq == dir)
29         return -1;
30     else if (x > dicio->v[meio].chave)
31         return DicionarioBinaria(dicio, meio+1, dir, x);
32     else if (x < dicio->v[meio].chave)
33         return DicionarioBinaria(dicio, esq, meio-1, x);
34     else
35         return meio;
36 }
```



```
38 /* encontra o índice da chave x no dicionário */
39 int DicionarioFind(Dicionario *dicio, Chave c) {
40     int i, esq, dir;
41     if (dicio->n == 0)
42         return -1;
43     esq = 0;
44     dir = dicio->n-1;
45     do {
46         i = (esq + dir) / 2;
47         if (c > dicio->v[i].chave)
48             esq = i + 1;
49         else
50             dir = i - 1;
51     }
52     while (c != dicio->v[i].chave && esq <= dir);
53
54     if (c == dicio->v[i].chave)
55         return i;
56     else
57         return -1;
58 }
```

Análise de Pesquisa Binária

Análise

- ▶ A cada iteração do algoritmo, o tamanho da tabela é dividido ao meio.
- ▶ **Logo:** o número de vezes que o tamanho da tabela é dividido ao meio é cerca de $\log n$.
- ▶ **Ressalva:** o custo para manter a tabela ordenada é alto: a cada inserção na posição p da tabela implica no deslocamento dos registros a partir da posição p para as posições seguintes.
- ▶ Consequentemente, a pesquisa binária não deve ser usada em aplicações muito dinâmicas.

Considerações Finais

Conclusão

- ▶ Foram vistas duas abordagens de pesquisa.
- ▶ Estrutura de dados melhores são necessárias para tornar a pesquisa mais eficiente.

Árvores de Pesquisa

Bibliografia

Bibliografia

Os conteúdos deste material, incluindo figuras, textos e códigos, foram extraídos ou adaptados de:

 Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein, Clifford.

Introduction to Algorithms.

The MIT Press, 2011.

Exercício

- ▶ Dada a sequência de números: 0 1 2 3 4 5 6 7 8 9 10.
- ▶ Faça uma pesquisa binária pelo elemento 9, apresentando a sequência de execução a cada passo (Teste de Mesa).